Project 2: Cache & Memory Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We profile the cache and memory hierarchy of an Intel i9-10900F with four purpose-built microbenchmarks and user-space hardware counters. Zero-queue pointer-chase latency, taken on each level's plateau by a stated rule, is 1.79 ns in L1 (5.02 cycles), 4.76 ns in L2 (13.3 cycles), 18.6 ns in the 20 MiB L3 and 109 ns in DRAM with 4 KiB pages (85.9 ns with 2 MiB pages). A loaded-latency probe running concurrently with up to nine streaming cores traces the throughput–latency curve directly: latency doubles at 28.8 GiB/s (91% of the 31.6 GiB/s peak), and Little's Law with the *measured* loaded latency puts 99.5 lines in flight at the knee, against 54.6 that the memory system actually serves—the rest is queueing. A known-concurrency chase shows one core keeps at most 8.71 random demand misses in flight, however many independent chains it runs. Counter-based AMAT fits explain the 41× random-SAXPY slowdown, and 2 MiB pages—verified as granted—remove the page-walk cost past the 4 KiB TLB reach. We close with the cache-reuse bandwidth pitfall, where a 16 KiB buffer reports 32.5 GiB/s of nonexistent "DRAM" bandwidth.

1 Introduction

Every memory reference on this machine is served by one of four levels that differ by more than two orders of magnitude in latency: a 32 KiB per-core L1 data cache, a 256 KiB per-core L2, a 20 MiB shared L3, and DDR4 DRAM. Which level a kernel hits—and how many independent references it keeps in flight—decides its performance far more than its instruction count. This report measures that hierarchy on the Intel i9-10900F with purpose-built microbenchmarks, reads user-space hardware counters through `perf_event_open`, and interprets the numbers with AMAT and Little's Law.

This report answers:

1. What is the zero-queue access latency of each level, and how do latency and bandwidth change with access pattern, access size and read/write mix? (Sections 4 to 6)
2. Where does added concurrency stop buying throughput and start costing latency, and does Little's Law hold with measured latency? (Section 7)
3. How do working-set size, cache misses and TLB misses show up in counters and runtime? (Sections 8 to 10)
4. Which benchmark mistake produces misleading numbers? (Section 11)

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	single-thread runs and the loaded-latency probe pinned to CPU 2 (a physical core, not CPU 0); multi-threaded runs place one thread per physical core; no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 2); FIO: fio-3.42; MLC: not available (not installed)
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
Memory configuration	63 GiB total as reported by the kernel (`MemTotal`); single NUMA node; DIMM count and speed not probed (needs root/`dmidecode`)
Env. snapshot	2026-10-21 19:52 EDT (each raw run directory also stores its own `env.txt`)

Summary of findings. Every number below is generated from the processed data (`data/summary.csv`, Section A). Zero-queue latency is 1.79 ns (5.02 cycles) in L1, 4.76 ns (13.3 cycles) on the L2 plateau, 18.6 ns in L3 and 109 ns (306 cycles) in DRAM; an independently built cross-check agrees within 0.32% on DRAM. Under load, the probe's latency reaches twice its unloaded value at 28.8 GiB/s, against a 31.6 GiB/s ceiling; beyond the knee, extra cores buy little bandwidth but a lot of latency. Little's Law with the measured loaded latency gives 99.5 lines in flight at the knee, while a single core keeps at most 8.71 random demand misses in flight—below its fill-buffer count—so several cores are needed to fill the pipe. Random SAXPY runs 41× slower per element once its footprint reaches DRAM, versus 6.1× for sequential order, and an AMAT model fitted to the counters attributes this to DRAM misses whose effective cost is 14.5 ns each—the zero-queue latency divided by 7.5-way memory-level parallelism. Finally, a one-line change of working-set size turns a 10.8 GiB/s DRAM result into a fictitious 32.5 GiB/s "bandwidth" that is really L1 reuse (Section 11).

2 Methodology

2.1 Platform

All measurements were taken on the desktop system in Table 1: a ten-core Intel i9-10900F (Comet Lake, Skylake-family core) with per-core 32 KiB L1d and 256 KiB L2, a shared 20 MiB L3, and DDR4 memory on one NUMA node. The table is generated by `scripts/collect_env.sh` on the test host; each raw-data directory also stores its own snapshot (`env.txt`) taken immediately before that experiment, so the settings of every figure are on record even if the machine changes later.

2.2 Tools

All benchmarks share the harness in `ACS-Code/common` (monotonic raw timer, CPU pinning) plus a project header `src/memprof.h` (page-policy allocation, ring generators, counters), and

raw per-repetition rows are reduced by `tools/summarize` (median and inter-quartile range). `scripts/derive.py` then computes every derived number quoted in the text (plateaus, knee, Little's Law, AMAT fit) into `summary.csv`; the report reads those values directly, so no number in the prose is copied by hand.

`mem_latency`	single-thread pointer-chase latency: working-set size, access size, node stride, page size, ring generator, timer, counters
`mem_bandwidth`	streaming bandwidth with T pinned threads: read/write/copy/triad/rwmix, access-size blocks, byte stride
`mem_loaded`	loaded latency: T load-generator cores (stream, rwmix, or k independent chases) plus a concurrent latency probe
`kernel_miss`	SAXPY loop whose footprint, order and page size vary, with counters
Counters	`perf_event_open(2)` called directly, user space only [3]
Intel MLC	not installed; substitute cross-checks in Section 3 [1]

`perf` (perf version 7.2.11-1) is installed, but `perf_event_paranoid` is 2, which allows only user-mode counting of a process's own events, and we want counts confined to the timed region of each repetition rather than to the whole process. The benchmarks therefore call `perf_event_open(2)` themselves (`exclude_kernel=1`, `exclude_hv=1`), enable the counters immediately before each timed loop and read them immediately after it. The events are cycles, instructions, L1D read misses, LLC read references and misses, and dTLB read misses (completed page walks). Events are opened independently so the kernel may multiplex them; counts are scaled by `time_enabled/time_running` and the smallest running fraction is recorded per row (`ctr_cover`). Intel MLC is the spec's suggested cross-check but is not installed and could not be installed for this study; `run.sh mlc` records that fact in its raw directory, and Section 3 describes the clearly labelled substitutes.

2.3 Controls

- **Pinning and isolation:** single-thread runs and the loaded-latency probe pinned to CPU 2 (a physical core, not CPU 0); multi-threaded runs place one thread per physical core. SMT: off. The machine is otherwise idle and experiments are run one at a time. All experiment data in this report were collected on 2026-10-21, 00:43–00:55, in one `run.sh all` invocation; the `clock_check` diagnostic ran the same day at 19:46.

- **Frequency:** governor performance, turbo disabled, so the core runs at its fixed 2.8 GHz base clock (equal to the TSC rate). Cycle figures are ns $\times$ 2.80 GHz. This conversion was checked against the hardware: user-mode cycles divided by wall time over the timed windows give a median of 2.77 GHz. The benchmarks also log an in-process add-chain frequency probe (`ghz` column), but it is *not* used: in 32% of runs it read about 2.41 GHz instead of 2.79 GHz, although the cycle counter shows 2.77 GHz during exactly those runs. The low readings are therefore an artifact of the probe, not a clock change: a diagnostic (`clock_check`) traces them to wall time lost to bursts of ACPI events on this host (Section 12). Converting with the probe would have understated DRAM latency by about 14% (264 instead of 306 cycles); see Section 12.

- **Pages:** every buffer is a 2 MiB-aligned anonymous mapping whose page size is chosen by `madvise` *before first touch*: `MADV_HUGEPAGE` for "huge" and `MADV_NOHUGEPAGE` for "4k". The second is essential because THP is always: without it a "4 KiB" baseline can silently receive huge pages. Each run records the `AnonHugePages` actually granted (from `/proc/self/smaps_rollup`).

- **Pre-faulting and warm-up:** every buffer is written once to fault in its pages, then the ring or stream is traversed (or one untimed repetition is run) before timing, so neither page faults nor cold-cache effects fall inside the timed region.
- **Repetitions:** 8 per point; figures show the median with inter-quartile-range error bars.
- **Timing:** each timed interval is calibrated to $\geq$40 ms to 200 ms, far above the timer resolution, so timer overhead is negligible. The loaded-latency windows read generator progress counters at the start and end of the *same* interval in which the probe runs, so throughput and latency describe the same moment.
- **Provenance:** each `data/raw/<experiment>/<timestamp>/` directory holds the raw rows, `env.txt`, `build.txt` (git commit, compiler, flags, THP and counter settings), `commands.txt` (every command line) and `validate.txt` (the sanity gate of Section 3, run immediately before the experiment).

3 Validation and Sanity Checks

The microbenchmarks have no reference output to diff, so `scripts/validate.sh` (also `make test`) gates them against physical invariants that must hold on any sane hierarchy. `run.sh` executes it immediately before every experiment and stores the full output as `validate.txt` in that experiment's raw directory, so each dataset carries its own evidence that the checks passed on the machine state it was measured in. The checks are:

- **Monotone levels.** Random pointer-chase latency grows strictly across levels: L1 $<$ L2 $<$ L3 $<$ DRAM (measured values in Table 2).
- **Prefetcher check.** Sequential DRAM "latency" is more than 2$\times$ below random DRAM latency, confirming the stream prefetcher is engaged for sequential access and defeated by the randomized chase.
- **Bandwidth ceiling.** Single-core L1-resident read bandwidth exceeds the DRAM-resident rate, and ten threads exceed one thread.
- **Loaded $>$ unloaded.** The `mem_loaded` probe is slower under four streaming cores than on an idle memory system—the minimum requirement for a loaded-latency instrument.
- **Frequency plausibility.** The in-process add-chain probe lands in 1 GHz to 6 GHz. This is only a coarse check: cycle conversions use the fixed 2.8 GHz clock, which the cycle counter confirms (Section 2).
- **Instrumentation (reported, not gating).** User-space counters open and return values; a 2 MiB-page request is actually granted (`AnonHugePages` $>$ 0) and the 4 KiB policy yields none.

Substitute for the Intel MLC cross-check. The project asks for Intel MLC runs as an external cross-check. MLC is not installed on the test host (not available (not installed)); `run.sh mlc` detects this, prints "Intel MLC not installed" and records it in `data/raw/mlc/<timestamp>/mlc_status.txt`. In its place we use two *substitute* cross-checks, which are internal and therefore weaker than an independent tool—they share our methodology (dependent pointer chase) but not our code path:

1. **Second latency method.** Every zero-queue size is measured twice: the primary ring is built with a 32-bit LCG and timed with `CLOCK_MONOTONIC_RAW`; the cross-check ring is built with xorshift64* (a different permutation, seed and range reduction) and timed with `rdtscp`. On the DRAM plateau they differ by 0.32% (Table 2 lists every level); `validate.sh` fails if they differ by more than 15%.

2. **Different program.** The $T = 0$ point of `mem_loaded` measures unloaded DRAM latency with a separately written, pointer-based ring in a different binary: 103 ns, against 109 ns from `mem_latency` at the same 256 MiB size. The 6% gap is larger than the repetition noise; the two rings differ in layout (the probe stores raw pointers, `mem_latency` stores indices) and in the order pages are first touched, which we did not investigate further.

An L1 latency of 5.02 cycles also matches the published 4–5 cycle load-to-use latency of this core, anchoring the ns→cycle conversion.

Timer and loop overhead are bounded by calibrating every timed interval to tens of milliseconds (Section 2); the chase loop body is a single dependent load, so no overhead subtraction is needed.

4 Zero-Queue Baselines

Hypothesis (from first principles, then compared with the data)

With a single dependent access in flight, latency should rise in steps at the L1 (32 KiB), L2 (256 KiB) and L3 (20 MiB) capacities. We expect roughly 4–5 cycles for L1, ∼12 cycles for L2, a few tens of cycles for L3, and ∼80 ns (hundreds of cycles) for DRAM, matching Skylake-family numbers.

Fixed	one thread pinned to CPU 2; random (Sattolo) pointer chase; one node per 64 B line
Varied	working-set size inside each level's plateau window; page size (DRAM only); method (primary vs. cross-check)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh zero_queue`

The ring is a single random cycle over one node per cache line, so each load depends on the previous one and exactly one request is outstanding—zero-queue latency. Cycles are converted with the fixed 2.8 GHz core clock, which the cycle counter confirms (Section 2).

Plateau rule. A level's latency is *not* read at its capacity edge, where the set is partly spilling (e.g. 256 KiB is a transition point, not an L2 point). Instead, `derive.py` takes the median, over all measured sizes s with $2\,C_{\text{prev}} \le s \le 0.75\,C_{\text{level}}$, of the per-size medians ($C_{\text{prev}} = 4$ KiB for L1); DRAM uses every size $\ge 4\times$ the L3. The same values feed the abstract, introduction and conclusion through generated macros, so they cannot disagree.

4.1 Analysis

Latency is flat at 1.79 ns (5.02 cycles) while the set fits in L1, sits at 4.76 ns (13.3 cycles) on the L2 plateau (individual sizes 4.31–6.35 ns; the high end is 192 KiB, where 4 KiB pages already cause L2 conflict misses, Section 8), spans 14.4–23.7 ns across the L3 (median 18.6 ns; L3 latency is not a single number because larger sets also miss the second-level TLB and travel further on the ring), and reaches 109 ns (306 cycles) in DRAM. With 2 MiB pages the DRAM figure drops to 85.9 ns: the 23.1 ns difference is the page walk that every random 4 KiB access beyond the TLB reach pays (Section 10), which is why MLC-style tools quote DRAM latency on huge pages. The cross-check

Table 2: Zero-queue random-access latency per level (median over the plateau sizes; each size is the median of 8 repetitions); cycles = ns × 2.8 GHz, the fixed core clock with turbo disabled (Section 2). "min–max" is the spread across plateau sizes. The cross-check is a *substitute* for Intel MLC (not installed): an independently generated ring (xorshift64*) timed with `rdtscp`. "2 MiB pages" removes the page-walk component from DRAM latency. Generated from `data/zero_queue_table.csv`.

Level	Plateau sizes	ns	cycles	min	max	x-check ns	Δ %	2 MiB pages
L1	8–24 KiB	1.79	5.02	1.79	1.80	1.79	0.0055	—
L2	64–192 KiB	4.76	13.3	4.31	6.35	4.77	0.26	—
L3	0.5–15 MiB	18.6	52.1	14.4	23.7	18.6	-0.26	—
DRAM	≥80 MiB	109	306	98.7	112	110	0.32	85.9

method differs by 0.0055%, 0.26%, -0.26% and 0.32% on L1, L2, L3 and DRAM. Against the hypothesis, L1 (5 cycles), L2 (13.3 vs. ∼12 cycles) and L3 match; DRAM with 4 KiB pages is about a third slower than the ∼80 ns we expected, and the 2 MiB-page value shows that most of the excess is the page walk.

Caveats: the L3 and DRAM figures are a single-core view of shared resources on an otherwise idle system; sequential "latency" (Section 5) is prefetcher throughput, not latency, which is why the chase is randomized; and both methods are pointer chases, so the cross-check guards against ring construction and timer errors but not against a shared methodological bias, which only an independent tool such as MLC could rule out.

5 Access Pattern and Granularity

Hypothesis (from first principles, then compared with the data)

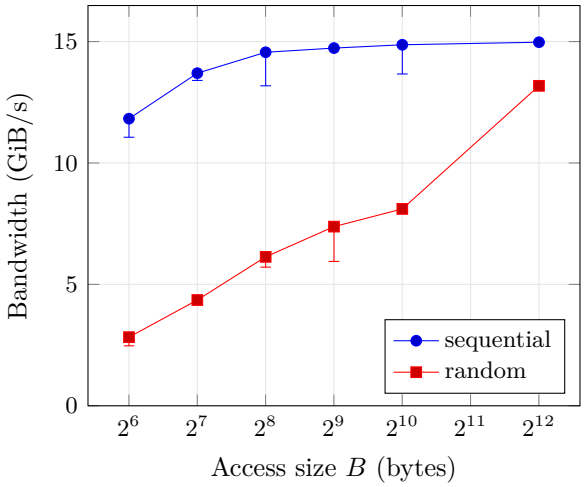
Access size. Reading a whole B-byte block per access should leave sequential bandwidth roughly unchanged (the stream is the same) but raise random bandwidth steeply with B, because a larger block amortises one DRAM latency over more lines and lets the adjacent-line and stream prefetchers help. The latency of one dependent B-byte access should grow far less than B, since its lines are fetched in parallel. *Stride.* With one 8 B element used per stride, useful bandwidth should fall roughly as 1/stride once the stride reaches a 64 B line.

Fixed	1 thread pinned to CPU 2; 256 MiB DRAM-resident region; 100% reads; 4 KiB pages
Varied	pattern (sequential, random); access size $B = 64 \ldots 4096$ B (primary); byte stride 8–1024 (secondary)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pattern`

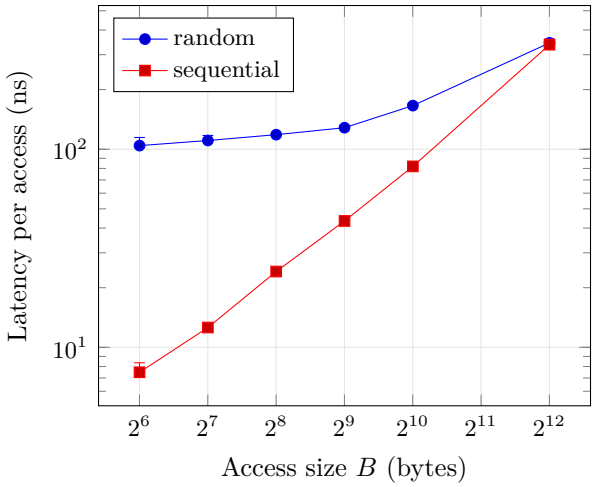
Definitions. *Access size B*: the buffer is cut into B-byte blocks and every byte of a block is read (256 bit vector loads) before the next block; sequential visits blocks in address order, random in a shuffled order. For *latency*, `mem_latency -access B` chains the blocks: the next block's address depends on all lines of the current block, so one access is one dependent B-byte request. *Stride* (secondary, Figure 2) keeps the earlier definition: one 8 B element used per stride, scalar FP loads.

5.1 Analysis: access size

At $B = 64$ B sequential blocks stream at 11.8 GiB/s and random blocks at 2.83 GiB/s (4.17× apart). Growing the block to 256 B and 1024 B lifts random bandwidth to 6.13 GiB/s and 8.11 GiB/s



(a) Streaming bandwidth of B-byte blocks.



(b) Latency of one dependent B-byte access.

Figure 1: Access pattern and access size, DRAM-resident, one core. Strided rows of `pattern_bw.csv` ($B = 0$) are dropped from the log axis and shown in Figure 2.

(sequential: 14.6 GiB/s, 14.9 GiB/s), shrinking the gap to 1.84×; at 4096 B random blocks reach 13.2 GiB/s, close to the sequential 15.0 GiB/s. Within a block lines are contiguous, so their misses overlap and the prefetchers engage. Sequential bandwidth is flat from 256 B up, as expected; it is lower only at 64 B, where the per-block loop overhead is paid on every line. The latency panel tells the same story per request. A random dependent 64 B access costs 104 ns, but a 1024 B one costs only 166 ns—16 lines for far less than 16 latencies—so a single dependent chain moves 5.74 GiB/s instead of 0.572 GiB/s. Sequentially chained blocks cost 7.48 ns (64 B) and 81.9 ns (1024 B); the difference from the random values is the part of the DRAM latency the stream prefetcher hides by fetching the next block early, so sequential "latency" is partly throughput. The hiding fades as blocks grow: at 4096 B a sequential access costs 337 ns, no better than random (344 ns). Every block then starts a new 4 KiB page, and the stream prefetcher does not prefetch across page boundaries, so each dependent access begins with a full miss.

5.2 Analysis: stride, and the line-traffic anomaly

Sequential useful bandwidth is 10.9 GiB/s at unit (8 B) stride and 1.96 GiB/s at 64 B. At 64 B stride every access still pulls a whole line, so the *line* traffic is 8× the useful bandwidth: 15.7 GiB/s, i.e. 1.44× the unit-stride rate. A unit-stride scalar read is therefore *not* a measure of the single-core streaming limit, although it is the obvious kernel to use for one. The explanation is on the core side, not in memory: the unit-stride scalar kernel spends eight loads and eight FP adds on every line, so fewer distinct lines fit in the out-of-order window and fewer misses are in flight. Kernels that spend fewer instructions per line move more lines: one load per line (64 B stride) gives 15.7 GiB/s, and full-line vector reads of large blocks give up to 15.0 GiB/s. The vector kernel at 64 B blocks, which still pays per-block loop overhead, is only 1.08× the scalar kernel (11.8 GiB/s). The single-core streaming limit is therefore about 15 GiB/s to 16 GiB/s, and no measured line traffic exceeds it. The loaded-latency study (Section 7) uses a two-loads-per-line vector stream. Contrary to our hypothesis, useful bandwidth does *not* keep falling as 1/stride beyond one line: it is 0.915 GiB/s at 256 B and 0.800 GiB/s at 1024 B, so touched-line traffic drops from 15.7 GiB/s to 7.32 GiB/s and 6.40 GiB/s. Once each access is a separate line, the kernel is limited by how many misses it keeps in

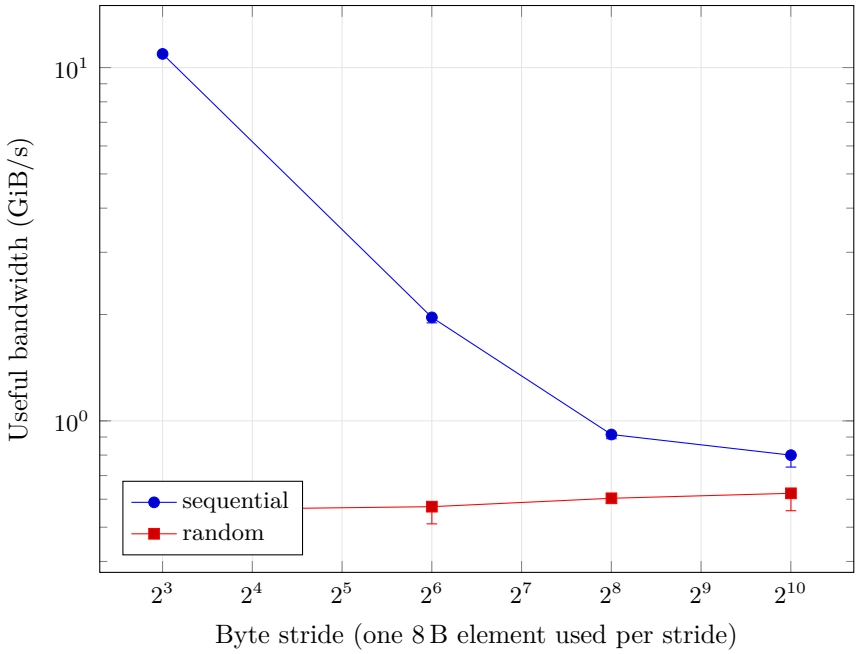


Figure 2: Secondary result: useful bandwidth vs. byte stride with scalar 8 B loads.

flight, not by bandwidth; skipping lines saves bandwidth it was not using. At 256 B and 1024 B strides the spatial (adjacent-line) prefetcher may fetch a second line per access, so DRAM traffic may exceed the touched-line traffic; without uncore counters (unavailable at `perf_event_paranoid`=2) we cannot measure that, and we report only touched-line and useful bandwidth. Random strided access is flat at 0.560–0.624 GiB/s: each access is an independent miss, so stride barely matters.

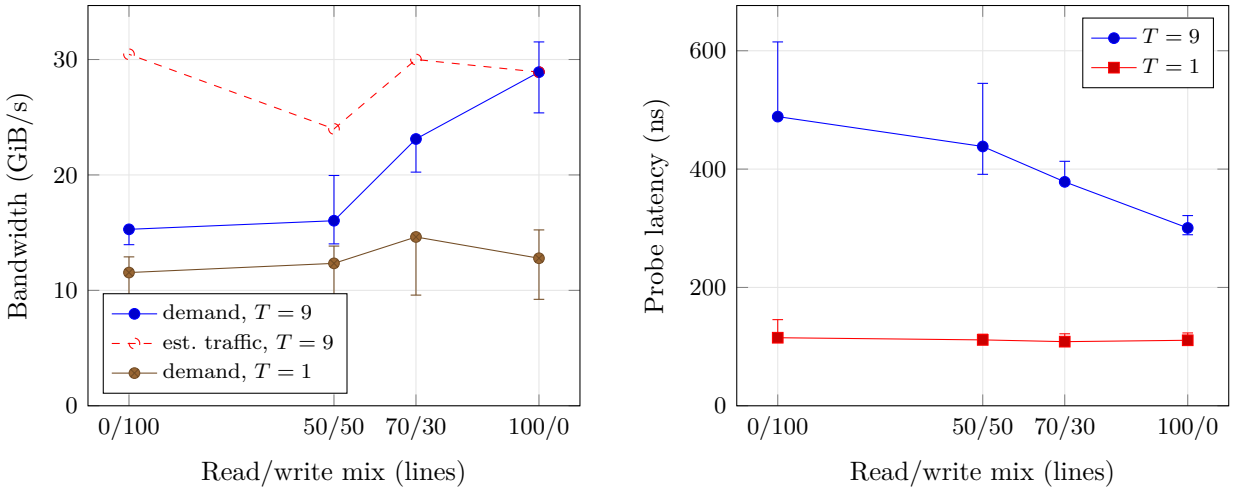
6 Read/Write Mix

Hypothesis (from first principles, then compared with the data)

Pure reads should reach the highest *demand* bandwidth. Every written line first needs a read-for-ownership (RFO) and later a write-back, so a written line costs about two lines of DRAM traffic and adds read/write bus turnarounds. Demand bandwidth should therefore fall as the write fraction grows, while estimated DRAM traffic stays closer to constant; and loaded latency should be highest for the write-heavy mixes, whose write-backs compete with the probe's reads.

Fixed	128 MiB per generator core; sequential; ordinary (write-allocate) full-line stores; 4 KiB pages
Varied	reads per 10 *lines*: 10 (100/0), 7 (70/30), 5 (50/50), 0 (0/100); $T = 1$ and 9 generator cores
Latency	concurrent random pointer-chase probe on CPU 2 (`mem_loaded`); $T = 0$ gives the unloaded value
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh rw_mix`

Mix definition. Of every 10 consecutive 64 B lines, the first R are read and the remaining $10 - R$ are fully overwritten. Mixing at *line* granularity makes the ratio hold for the traffic the memory system sees. Mixing individual 8 B elements instead is a methodological trap: with 7 reads and 3 writes per 10 elements, every line of the source is read *and* every line of the destination written, so a nominal "70/30" moves about three lines per useful line and measures the access pattern, not the



(a) Throughput.

(b) Loaded latency of a random read.

Figure 3: Read/write mixes: throughput and concurrently measured latency. The unloaded probe latency is 103 ns.

mix (Section 13). "Demand" bandwidth counts the bytes the program reads or writes; "est. traffic" adds one RFO line per written line (written lines count twice).

6.1 Analysis

With nine generator cores, demand bandwidth is 28.9 GiB/s for 100% reads, 23.1 GiB/s at 70/30, 16.0 GiB/s at 50/50 and 15.3 GiB/s for 100% writes. Counting the RFO, estimated DRAM traffic is 28.9 GiB/s, 30.0 GiB/s, 24.0 GiB/s and 30.5 GiB/s respectively. Traffic is therefore roughly constant at the controller's ceiling for pure reads, 70/30 and pure writes—each written line costs its demand line plus one RFO, and the controller delivers about the same number of lines either way—but drops by about a fifth at 50/50, where the bus has to turn around between reads and writes most often. Demand bandwidth falls monotonically with the write fraction, as predicted. The concurrently measured probe latency is 301 ns, 378 ns, 438 ns and 489 ns for the same four mixes (unloaded: 103 ns); write-backs occupy the same channels and queues as the probe's reads, so a write-heavy stream raises read latency even at lower demand bandwidth. A single generator core ($T = 1$) is not controller-bound and behaves differently: its demand bandwidth is 12.8 GiB/s for reads, *highest* at 70/30 (14.6 GiB/s) and 11.5 GiB/s for writes, while its estimated traffic grows from 12.8 GiB/s to 22.6 GiB/s. A plausible explanation, not tested here, is that stores retire into the store buffer without waiting for their RFO, so a core can keep more lines moving when part of its stream is writes than when every line is a load it must wait for. The probe barely notices one core: 108–115 ns across the four mixes. As named-kernel references on all ten cores, `copy` delivers 15.8 GiB/s and `triad` 15.7 GiB/s (demand bytes; 1:1 and 2:1 read:write). Non-temporal stores, which skip the RFO, were out of scope for the required mixes.

7 Intensity Sweep and the Throughput–Latency Knee

Hypothesis (from first principles, then compared with the data)

A single core cannot saturate the DRAM controller because it can keep only about a dozen demand misses in flight (its L1 fill buffers). Adding load should raise delivered bandwidth until the dual-channel controller saturates in the low tens of GiB/s; past that point extra load should buy almost no bandwidth but sharply raise the latency of every request (queueing). By Little's Law, saturation needs tens of outstanding lines—more than one core provides.

Generators	`mem_loaded`: $T = 0 \ldots 9$ cores (all physical cores except CPU 2), each streaming its own 128 MiB region with 256 bit loads; then $T = 9$ throttled by an injection delay of 8–2048 empty iterations per line
Probe	random pointer chase over 256 MiB on CPU 2, measured in the same window as the generators' throughput
Known MLP	T cores each running k independent random chases: $T = 1$, $k = 1 \ldots 32$; $T = 2 \ldots 9$, $k = 8$
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh intensity`

Method. Each point reports *both* axes from one measurement window: generator threads publish progress counters, the main thread snapshots them, runs the latency probe for 200 ms, and snapshots again. Throughput is all lines moved (generators plus probe) per second; latency is the probe's time per dependent access—a demand read issued *under* that load, i.e. MLC's "loaded latency". $T = 0$ is the unloaded latency of the same setup ($W_0 = 103$ ns).

Knee criterion. We define the knee objectively as the throughput at which loaded latency first reaches $2W_0$, walking the thread sweep in order of *offered* load ($T = 0, 1, \ldots, 9$) and interpolating linearly between the two bracketing points (`derive.py`). The same rule applied to the injection-delay sweep (decreasing delay at $T = 9$) is a second estimate, and the point of maximum throughput/latency is a third. All three come from the CSV, not from the eye.

7.1 Knee and diminishing returns

The curve has the textbook shape (Figure 4, Table 3). From $T = 0$ to $T = 4$ bandwidth climbs from the probe's own trickle to 30.6 GiB/s—already 95% of the 31.6 GiB/s maximum—while latency rises only from 103 ns to 142 ns (1.38×). Every core added after that buys no bandwidth: delivered bandwidth wanders between 27.9 GiB/s and 31.6 GiB/s (run-to-run noise; see the IQR in Figure 4), but latency climbs steadily to 305 ns at $T = 9$; at the bandwidth maximum it is already 2.50× the unloaded value. By the $2W_0$ rule the knee lies at 28.8 GiB/s (91% of peak), between $T = 5$ and $T = 6$. The delay sweep crosses $2W_0$ at 30.8 GiB/s (between delays 16 and 8), and the throughput/latency maximum is at $T = 4$ (30.6 GiB/s, 142 ns). The three estimates agree that the useful operating range ends at four to six streaming cores and 29 GiB/s to 31 GiB/s. Beyond it the dual-channel controller, not the cores, is the bottleneck: requests no longer finish faster, they queue.

7.2 Little's Law with measured latency

For every load level we compute the mean number of lines in flight, $L = \lambda W$ [2], with λ the delivered line rate and W the *measured* loaded latency of that same window. For comparison, $L_0 = \lambda W_0$ uses the unloaded latency, the usual shortcut when loaded latency is not measured. L_0 rises to 54.6 lines at saturation and then stays flat: that is the concurrency the memory system can actually *serve*

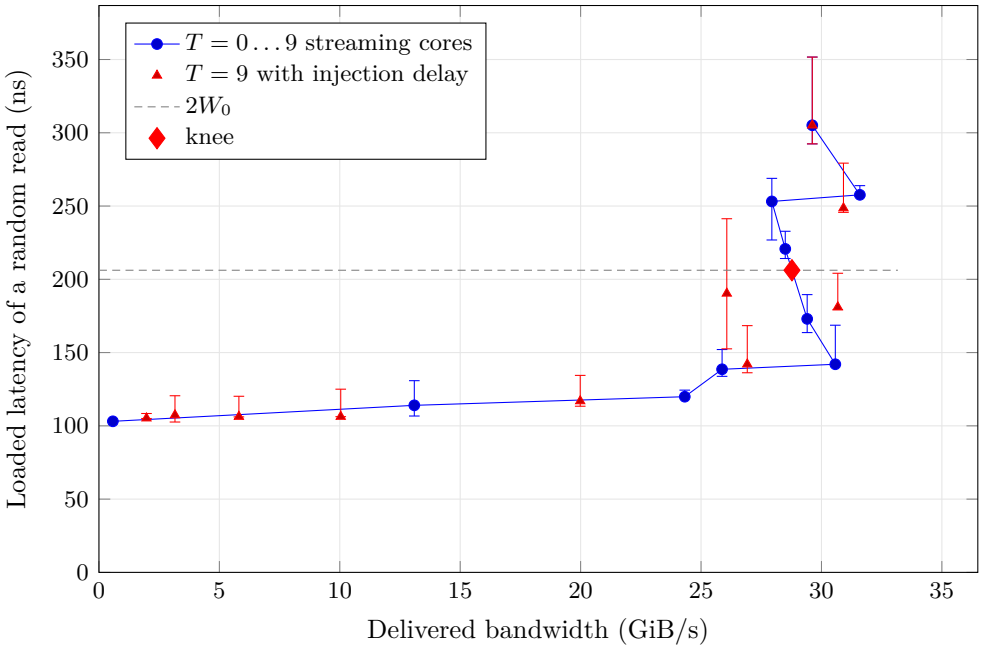


Figure 4: Measured throughput–latency curve. Each point is one load level; both coordinates are medians of 8 windows (bars: IQR of latency). The knee is where latency first reaches twice its unloaded value.

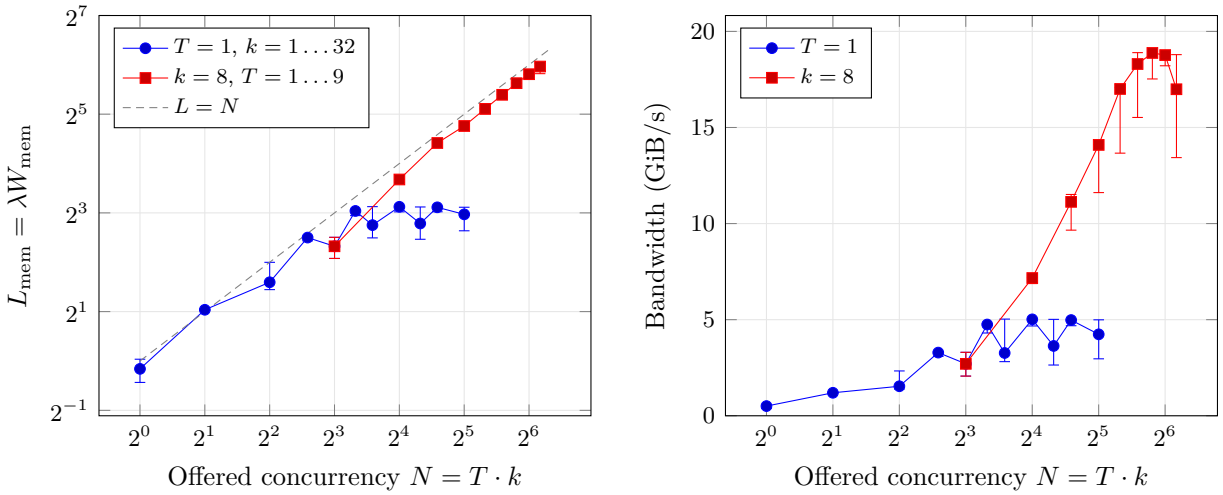
at about 30 GiB/s. L, in contrast, keeps growing to 152 lines at $T = 9$; the difference $L - L_0$ is requests waiting in queues, which is exactly why latency rises without any bandwidth gain. At the knee, $L = 99.5$.

What the known-concurrency sweep shows. Computing $L = \lambda W$ with W taken as each chain's own step time gives L/N between 1.00 and 1.02. That is *not* a validation of Little's Law: with that W, $\lambda W = N$ holds by construction, so it only confirms the progress accounting. The informative quantity is $L_{\text{mem}} = \lambda W_{\text{mem}}$, where W_{mem} is the latency of a memory request measured at the same time by the probe (107 ns on average with one generator core). One core reaches its best random-read bandwidth, 5.02 GiB/s, at $k = 16$, and its L_{mem} never exceeds 8.71 lines (Figure 5). Chains beyond that do not reach memory any sooner: they wait inside the core, so the per-chain step time grows instead (to 465 ns at $k = 32$). About nine outstanding demand misses per core is consistent with the 12 L1D fill buffers of this Skylake-family core. The single-core curve is not monotonic in k ($k = 8$ and 12 are slower than $k = 6$ and 10); we did not resolve this generator artifact and use only the envelope. With $k = 8$ per core, adding cores raises bandwidth to 18.9 GiB/s at $T = 7$ ($L_{\text{mem}} = 49.4$, $W_{\text{mem}} = 156$ ns). Random demand reads thus reach only about 60% of the streaming peak: without prefetching, nine cores of this kind keep about seven lines each in flight and the memory system queues long before it is fully used.

Reconciling one core's streaming concurrency with the fill buffers. The streaming sweep gives one core $L = 25.0$ lines in flight at 13.1 GiB/s (Table 3), almost three times the 8.71 demand misses one core sustains. There is no contradiction: in a sequential stream most lines are brought in by the L2 hardware prefetcher, whose requests are tracked in the L2's own request queue rather than in the L1 fill buffers, so a streaming core keeps more lines in flight than a core issuing random

Table 3: Little's Law across the thread sweep (no injection delay), generated from `data/littles_law.csv`. λ = delivered lines/s (64 B lines), W = loaded probe latency, $L = \lambda W$; $L_0 = \lambda W_0$ with the unloaded W_0.

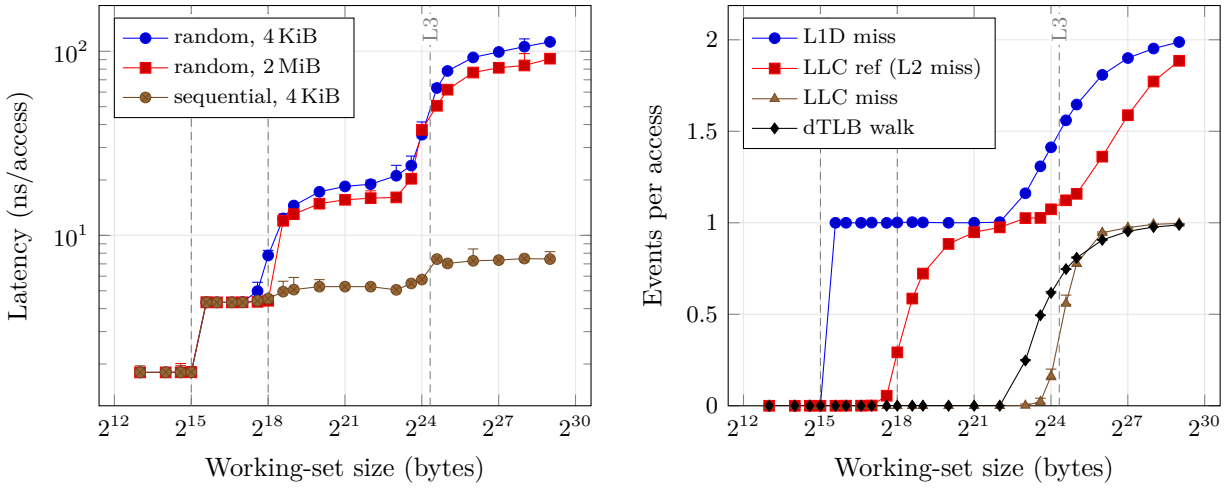
T	BW (GiB/s)	λ (Mlines/s)	W (ns)	$L = \lambda W$	$L_0 = \lambda W_0$
0	0.58	10	103.1	1.0	1.0
1	13.09	220	114.0	25.0	22.6
2	24.32	408	119.9	48.9	42.1
3	25.87	434	138.6	60.2	44.8
4	30.58	513	142.0	72.9	52.9
5	29.41	493	173.0	85.4	50.9
6	28.50	478	220.8	105.5	49.3
7	27.94	469	253.1	118.7	48.3
8	31.59	530	257.7	136.6	54.6
9	29.62	497	305.1	151.6	51.2



(a) Lines in flight in memory vs. offered N.

(b) Random-read bandwidth vs. N.

Figure 5: Known-concurrency sweep: k independent random chases per core (median of 8 windows, bars: IQR). L_{mem} uses the memory latency W_{mem} measured by the concurrent probe; the dashed line is $L = N$, which would be reached only if every chain always had a request in the memory system.



(a) Latency.

(b) Counters, random chase, 4 KiB pages.

Figure 6: Working-set sweep. Left: latency with cache capacities marked. Right: user-space counters per access for the random 4 KiB-page chase; each step in latency coincides with a miss ratio rising towards one.

demand misses. Quoting the fill-buffer count as the per-core limit is correct only for demand misses; the two measurements quantify both limits. The 54.6 lines the controller serves at saturation are about two streaming cores' worth of concurrency (with queueing, four cores are needed to reach 95% of peak), but six random-access cores' worth.

8 Working-Set Sweep

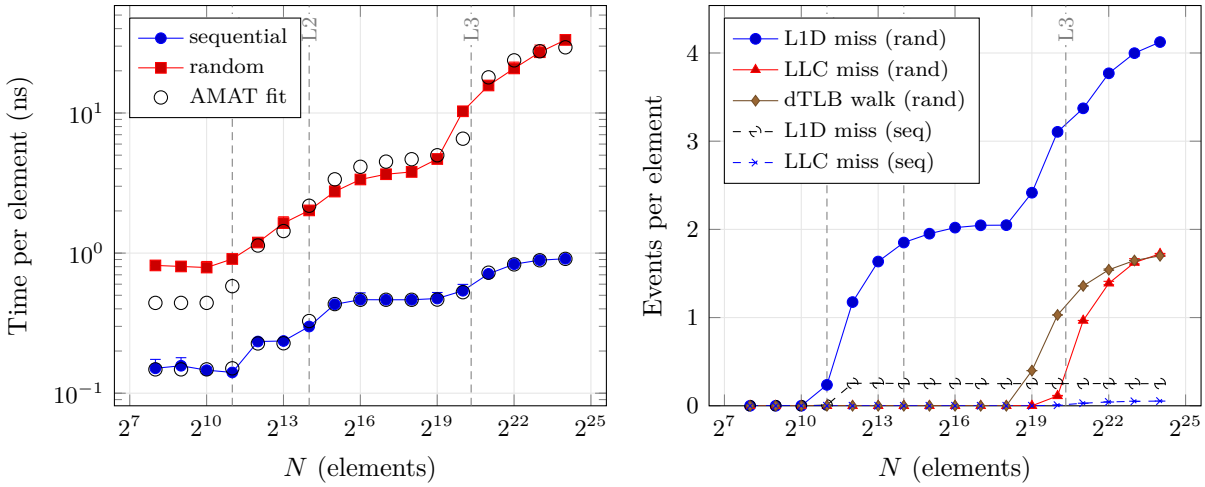
Hypothesis (from first principles, then compared with the data)

Pointer-chase latency should trace the capacity staircase: flat while the set fits in L1 (32 KiB), a step up through the 256 KiB L2, a wider plateau across the 20 MiB L3, and a final jump to DRAM latency. The counters should show the cause: L1D misses per access rising from 0 to 1 past 32 KiB, LLC misses per access rising to 1 past 20 MiB, and dTLB walks appearing once the set exceeds the 4 KiB-page TLB reach (∼6 MiB) but not with 2 MiB pages.

Fixed	1 thread pinned to CPU 2; one node per 64 B line; user-space counters
Varied	working-set size 8 KiB–512 MiB; random vs. sequential; 4 KiB vs. 2 MiB pages (random)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh working_set`

8.1 Analysis

The latency staircase and the counters line up level by level (Figure 6). L1D misses per access jump from ≈0 to 1 between 32 KiB and 48 KiB, exactly at the L1 capacity. LLC references (L2 misses) start rising at 192 KiB, before the 256 KiB capacity, and approach one per access only by 1 MiB to 2 MiB. LLC misses appear from 12 MiB to 16 MiB, also before the nominal 20 MiB, since the L3 is shared and page-table lines compete for it, and reach one per access by 64 MiB. Page walks start at 8 MiB, just past the ≈6 MiB STLB reach, and explain why latency already climbs inside the L3 region. In DRAM the random 4 KiB-page chase incurs 0.988 completed page walks and 1.99 L1D



(a) Runtime and fitted AMAT model.

(b) Counters per element.

Figure 7: SAXPY time per element and counters vs. footprint and traversal order (two `double` arrays, $16N$ bytes; lines mark where $16N$ crosses L1/L2/L3). Open circles: AMAT model fitted to the counters (Equation (1)).

misses per access (versus 1.00 with 2 MiB pages): the page walker's own page-table reads miss the L1 too, which is why the 4 KiB curve sits above the 2 MiB curve (112 ns vs. 91.2 ns at 512 MiB). The sequential chase costs only 7.43 ns in DRAM with 0.0386 demand LLC misses per access versus 0.996 for random: the prefetcher turns its misses into hits.

Transitions are gradual rather than sharp. At 192 KiB (75% of L2) the 4 KiB-page chase already costs 4.98 ns with 0.0548 L2 misses per access, while the 2 MiB-page chase stays at 4.36 ns (0.00254): about 20 times fewer L2 misses. The likely explanation is page placement: 4 KiB pages scatter the buffer over arbitrary physical frames, so some L2 sets receive more lines than their associativity allows (conflict misses), whereas a physically contiguous 2 MiB page spreads lines evenly over the sets. This staircase is the input to the AMAT model in Section 9. Counter multiplexing was in effect for some events (minimum running fraction 0.662); counts are scaled accordingly.

9 Cache-Miss Impact

Hypothesis (from first principles, then compared with the data)

As the SAXPY footprint grows past each cache, time per element should rise in proportion to the misses per element at each level. Sequential traversal should rise gently—the prefetcher turns misses into hits—while a random permutation should rise sharply once the set leaves cache. Because independent iterations overlap their misses, the effective cost of a DRAM miss should be well below the zero-queue DRAM latency.

Kernel	$y_i = a\,x_i + y_i$ (float64), two arrays, 4 KiB pages
Varied	$N = 2^8 \ldots 2^{24}$ (footprint $16N$ bytes); order (sequential, random permutation)
Counters	L1D read misses, LLC read references/misses, dTLB walks, cycles, instructions (user space, per element)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh cache_miss`

9.1 Analysis

Sequential SAXPY rises from 0.151 ns per element in L1 to 0.911 ns in DRAM, a 6.1× span, with only 0.0534 demand LLC misses per element at the largest size: the prefetcher has already brought most lines in. Random traversal rises from 0.817 ns to 33.2 ns, a 41× span, with 4.12 L1D misses, 1.72 LLC misses and 1.70 page walks per element at $N = 2^{24}$ (each element touches a line of x, a line of y, plus page-walk reads). At DRAM sizes the random curve is 36× the sequential one: that gap *is* the miss cost the prefetcher hides for regular code.

9.2 AMAT fitted to measured miss counts

Classic AMAT, $t_{L1} + m_{L1}(t_{L2} + m_{L2}(t_{L3} + m_{L3}\, t_{\mathrm{DRAM}}))$, assumes misses are serviced one at a time. We instead fit its per-element form to the counters, separately for each order:

$$t_{\mathrm{elem}} = t_0 + a_2\, h_{L2} + a_3\, h_{L3} + a_D\, m_D, \tag{1}$$

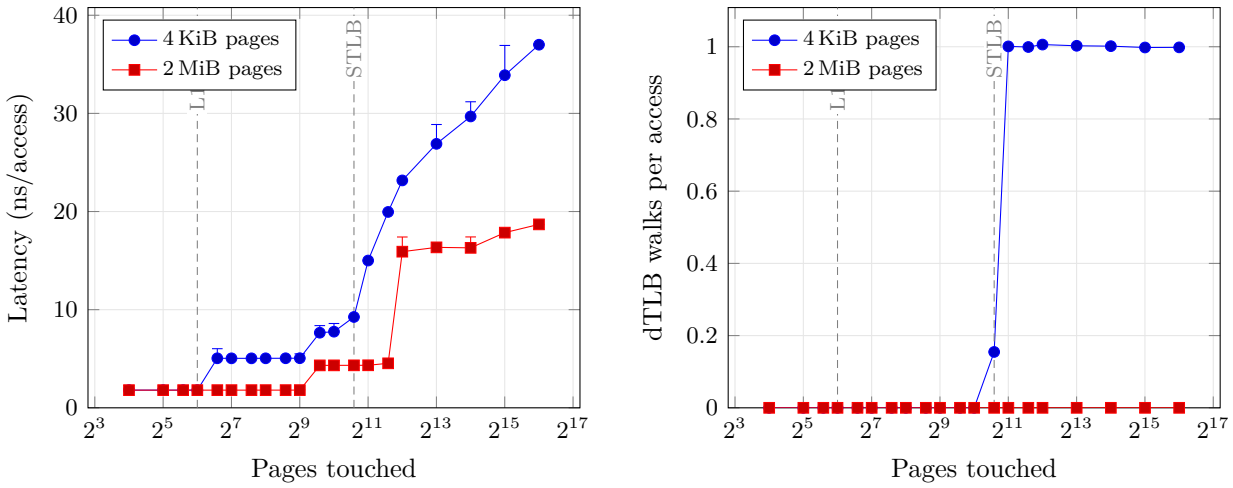
where h_{L2} = L1D misses − LLC references (served by L2), h_{L3} = LLC references − LLC misses (served by L3) and m_D = LLC misses (served by DRAM), all per element; t_0 absorbs the L1-hit work. Ordinary least squares over the 17 footprints gives, for random order, $t_0 = 0.441$ ns, $a_2 = 0.588$ ns, $a_3 = 2.10$ ns, $a_D = 14.5$ ns ($R^2 = 0.973$); for sequential order $a_D = 8.49$ ns ($R^2 = 0.999$). The fitted a_D is the *exposed* cost of one DRAM miss. Dividing the zero-queue DRAM latency by it gives the average number of DRAM misses overlapped: 7.5 for random and 13 for sequential SAXPY. For random order the same reading holds at every level: the zero-queue L2 and L3 latencies are 8.1× and 8.8× the fitted a_2 and a_3. All three overlap factors (7.5–9) match the 8.71 demand misses one core keeps in flight in Section 7: independent SAXPY iterations fill the same fill buffers that the chase experiment measured, and AMAT with serialised misses would overpredict the DRAM-resident runtime about 7.5-fold. Sequential order overlaps more (13 for DRAM) because the misses that remain after prefetching arrive in long, predictable runs. Caveats: all coefficients came out positive, but the fit is a four-parameter regression on 17 footprints, so the individual L2/L3 terms are only roughly determined; the random-order fit has its largest relative error, 46%, at the smallest and mid-size footprints, where one shared t_0 cannot describe the index-load and loop overhead (sequential: 9.5%). The page-walk cost is folded into a_D because walks and DRAM misses grow together in this sweep (Section 10 separates them).

10 TLB-Miss Impact

Hypothesis (from first principles, then compared with the data)

With 4 KiB pages, a dependent access to a random page should cost extra once the number of pages touched exceeds the L1 dTLB (64 entries) and again past the second-level TLB (1536 entries, ≈6 MiB of reach), where every access needs a page walk. With 2 MiB pages one entry covers 512× more memory, so the same page counts should need no walks and the extra latency should vanish.

Chase	one node per 4 KiB page (node stride 4160 B = page + one line, so nodes fall into different cache sets); random order; 16–65536 pages
SAXPY	random-order $y_i = a\, x_i + y_i$, $N = 2^{18} \ldots 2^{24}$
Varied	page size: 4 KiB (`MADV_NOHUGEPAGE`) vs. 2 MiB (`MADV_HUGEPAGE`), both before first touch
Verification	`AnonHugePages` per run; dTLB-walk counter per access
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh tlb`



(a) Page-stride chase latency.

(b) Completed page walks (counter).

Figure 8: TLB reach: dependent access to one line per page. The gap between the curves is the cost of TLB misses; the counter confirms walks occur only with 4 KiB pages past the STLB reach.

The page-stride chase is designed so that page size is the *only* difference between the two curves: both touch the same number of cache lines (one per page, so the cache footprint is N lines), and the one-line stagger avoids every node mapping to the same cache set. Any latency gap is therefore TLB reach and page-walk cost.

10.1 Analysis

The curves show both TLB levels as separate cliffs (Figure 8). Up to 64 pages both page sizes cost the same (1.80 ns vs. 1.80 ns). At 128 pages the 4 KiB chase jumps to 5.04 ns while the 2 MiB chase stays at 1.80 ns, with zero page walks in both. The 3.24 ns (9.07 cycles) gap is the cost of missing the 64-entry L1 dTLB and hitting the second-level TLB. Near the STLB capacity walks begin (0.155 per access at 1536 pages), and at 2048 pages every 4 KiB access walks (1.00 walks per access): 15.0 ns against 4.33 ns, a 10.7 ns (29.9-cycle) page-walk penalty while the page-table entries still hit in cache. Past 4096 pages the cache footprint itself leaves L2 (4096 lines = 256 KiB), raising both curves—the huge-page chase rises to 15.9 ns—so only the gap is attributed to the TLB. That gap grows again as the page tables outgrow the caches: 13.4 ns at 16384 pages and 18.3 ns at 65536 pages (37.0 ns vs. 18.7 ns, 1.98×). With 2 MiB pages the walk counter stays at 0 even at 65536 pages.

For random SAXPY at $N = 2^{24}$, 2 MiB pages cut page walks from 1.70 to 0.00000132 per element and runtime from 31.8 ns to 17.0 ns per element, a 1.87× speedup, close to the 1.98× of the dependent chase. Overlapping independent misses therefore does not hide the walks: each walk adds dependent page-table reads in front of its data miss.

Huge pages are verified, not assumed. Requesting huge pages does not guarantee them: a benchmark that only asks for THP can silently run on 4 KiB pages and report "no effect" (Section 12). The benchmarks map 2 MiB-aligned regions and call `madvise` before the first touch; each row records the granted `AnonHugePages` (summarised in the run's `anon_thp.txt`), `validate.sh` checks the grant before each experiment, and the dTLB-walk counter independently confirms the page size in effect. In this run every "huge" configuration was fully backed: `AnonHugePages` equals the buffer

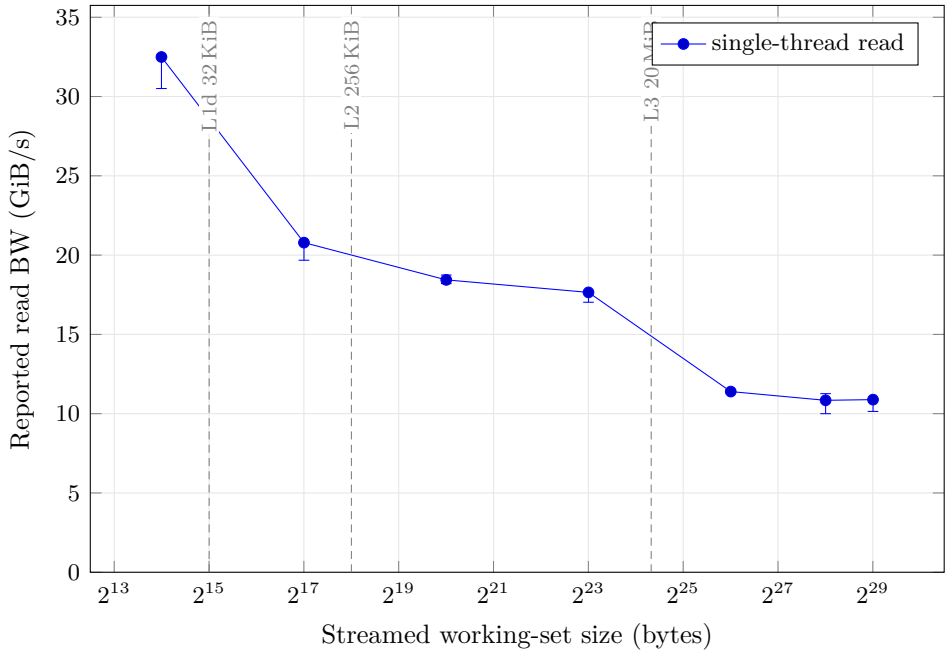


Figure 9: The same streaming kernel "measures" wildly different bandwidth depending only on the buffer size. A 16 KiB buffer reports 32.5 GiB/s; the honest DRAM-resident figure is 10.8–11.4 GiB/s. Nothing about the code changed—only where the data lived.

size at every point (for example 266 240 KiB for the 65536-page chase, and 327 680 KiB for SAXPY at $N = 2^{24}$, i.e. x, y and the index array), and every 4 KiB row records 0 KiB.

11 Benchmark Pitfall

Hypothesis (from first principles, then compared with the data)

A "DRAM bandwidth" benchmark that re-streams a buffer small enough to live in cache will report a number several times higher than the real memory bandwidth, because every pass after the first hits the cache instead of DRAM. We expect a buffer that fits in L1/L2 to report tens of GiB/s and a DRAM-resident buffer to settle near the single-core streaming figure from Section 5.

Pitfall	Measuring cache reuse instead of DRAM: picking a buffer that fits in cache
Wrong setup	Working set 16 KiB (16 320 B after rounding to whole loop units; L1-resident), re-streamed many times, one thread
Right setup	Working set 64 MiB to 512 MiB (well past the 20 MiB LLC)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pitfall`

11.1 Explanation

At 16 KiB the kernel reports 32.5 GiB/s, falling as the set spills L1 and L2; once the set is 64 MiB or larger—genuinely DRAM-resident—it settles at 10.8–11.4 GiB/s, a 3.0× overstatement for the small buffer. (This kernel is the scalar unit-stride read of Section 5; its DRAM figure is window-limited and is not the single-core peak.) The inflated small-buffer number is not DRAM bandwidth at all: it is L1/L2 bandwidth, reported by a loop that re-reads the same cached lines. An AI-generated

bandwidth microbenchmark very easily falls into this trap, because a 16 KiB array is a perfectly natural default and the resulting tens of GiB/s look like a plausible "memory" number. Two cheap detectors flag it: (1) the reported bandwidth *changes with buffer size*, which a true DRAM measurement must not once the set exceeds the LLC; and (2) the small-buffer figure exceeds what the whole memory system delivers to *all* cores (31.6 GiB/s, Section 7). Our methodology avoids the pitfall by sizing every bandwidth and latency point explicitly against the cache capacities (Section 2) and, for the DRAM claims, using working sets at least 3× the LLC so the first-touch pass cannot leave the data cached.

12 Anomalies and Limitations

12.1 Unexpected results and how they were resolved

- **The in-process frequency probe misreads the clock.** With turbo disabled, the add-chain probe logged by every run reads either 2.79 GHz or about 2.41 GHz, which would understate DRAM latency by about 14% in cycles. Hypothesis: the clock does not change, but the thread loses wall time. Test: `clock_check` (`run.sh clock_check`) compares core cycles, reference cycles and wall time in 80 windows of 330 ms on CPUs 2 and 6. The core clock is 2.79 GHz in every window. In 39% of the windows there is a burst of ACPI general-purpose events (about 114 per window, from a long-running ACPI notify storm on this host). In those windows the pinned thread executes user code for only 0.866 of the wall time (quiet windows: 0.998), with no system time or context switches accounted to it, and the probe reads 2.42 GHz (quiet: 2.79 GHz). The lost time is therefore invisible to the OS, which is consistent with firmware (SMM) handling of the events and affects every core. Resolution: cycles are converted with the fixed 2.8 GHz clock, and the probe is not used (Section 2).

- **Unit-stride "peak" versus strided line traffic.** At 64 B stride the line traffic (15.7 GiB/s) exceeds the unit-stride scalar read (10.9 GiB/s), the kernel one would naturally use as the single-core peak. Hypothesis: the scalar kernel is limited by the core's out-of-order window, not by DRAM. Test: large full-line vector blocks reach 15.0 GiB/s on the same data, so the single-core limit is 15 GiB/s to 16 GiB/s (Section 5).

- $L/N = 1$ **is not a test.** In the known-concurrency sweep, λ times the per-chain step time equals N by construction. The informative quantity is L_{mem} with the probe's memory latency, which saturates at 8.71 lines per core (Section 7).

- **Bandwidth is non-monotonic past saturation.** With 5–9 streaming cores, delivered bandwidth scatters between 27.9 GiB/s and 31.6 GiB/s with wide IQRs, while latency rises steadily. A knee rule that orders points by delivered bandwidth is therefore unstable; ours follows offered load.

- **Single-core chase bandwidth alternates with** k**.** $k = 8, 12, 20$ chains are slower than $k = 6, 10, 16$. Probe latency is unchanged, so the cause is in the generator loop, not in memory. Unresolved; only the envelope is used.

- **Two unloaded DRAM latencies differ by 6%.** The `mem_loaded` probe gives 103 ns and `mem_latency` gives 109 ns, both at 256 MiB with 4 KiB pages. The identical `mem_latency` configuration measured 106 ns in the `working_set` run a few minutes later, so half of the gap is run-to-run spread. That spread is consistent with repetitions overlapping the ACPI bursts above, which inflate a window by up to ∼15%. We did not isolate the rest; candidates are the

ring layout (raw pointers vs. indices with a scaled addressing mode) and the physical-page placement of each allocation.

- **Element-level read/write mixes measure the wrong thing.** Mixing 8 B elements reads every source line and writes every destination line, so a nominal 70/30 mix moves about three lines per useful line; mixes must be defined on lines (Section 6).
- **Requested huge pages are not necessarily granted—and vice versa.** An early TLB run requested THP and received `AnonHugePages: 0 kB`, so its "huge" curve was really 4 KiB pages. Conversely, with THP set to always a 2 MiB-aligned "4 KiB" buffer is backed by huge pages unless the program opts out. Both page policies are therefore set explicitly before first touch and verified per run (Section 10).

12.2 Limitations

- **Lost wall time from ACPI activity.** The host has a long-running ACPI notify storm that takes about 13% of the wall time from every core in 39% of 0.3 s windows (above). We cannot stop it (root needed). Each timed repetition lasts 40 ms to 200 ms, so some repetitions are inflated by up to that amount. Medians over 8 repetitions limit the effect, and the IQR bars show it; the cycle counter over the timed windows gives a median of 2.77 GHz of user cycles per wall nanosecond, i.e. typical timed windows were not affected. Rejecting repetitions whose reference-cycle/wall ratio is below 0.95 would remove the effect entirely, but this filter was added after the data were taken.
- **Intel MLC was unavailable.** It is not installed on the test host; `run.sh mlc` records this in `data/raw/mlc/<timestamp>/mlc_status.txt`. The cross-checks in Section 3 are clearly labelled substitutes: they validate ring construction, timing and code path, but share our pointer-chase methodology.
- **Core-only counters.** At `perf_event_paranoid`=2 only user-space core events of our own process are available: no uncore/IMC counters (actual DRAM traffic, prefetch fills, write-backs) and no kernel-mode events. DRAM traffic for write mixes is therefore an estimate (one RFO per written line), and prefetcher traffic is inferred. Generic LLC events map to offcore-response events on this core and may undercount prefetch-driven fills. Counters were multiplexed when more events were requested than free hardware counters; counts are scaled and the running fraction is recorded.
- **Loaded latency is measured with a probe that is itself load.** The probe adds one line in flight (included in throughput) and measures a demand read; the streaming generators' prefetches may see a different latency, so $L = \lambda W$ for the streaming sweep assumes the probe's latency is representative. The known-concurrency chase sweep does not need that assumption.
- **One machine and one memory configuration.** All numbers come from a single i9-10900F on one NUMA node; DIMM population and speed could not be read without root. Frequency policy for the runs: governor performance, turbo disabled, SMT off.
- **Report build backend.** The host has `bibtex` but not `biber`, so `biblatex` uses `backend=bibtex`.

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code throughout: with Claude Opus 5 for the first implementation (2026-10-12 to 10-20), and with Claude Opus 5.5 for the rework of 2026-10-21 that produced every measurement in this report. GCC gcc (GCC) 16.2.1 20260810 was used for the builds. The AI wrote the code and the prose. Claude Code also ran the timed experiments, on the user's machine (Table 1) and with the user's approval, after the user had set the performance governor, disabled turbo and taken the SMT siblings offline (these settings need root). Every number comes from those runs.

What they helped generate

The four microbenchmarks—`mem_latency` (Sattolo pointer chase with access-size, page-size and counter options), `mem_bandwidth` (multi-thread streaming with read/write/copy/triad/rwmix and access-size modes), `mem_loaded` (loaded-latency probe with streaming, read/write and known-concurrency generators) and `kernel_miss` (footprint-varying SAXPY with counters)—plus the shared helpers, the `perf_event_open` counter code, the experiment driver (`run.sh`), `validate.sh`, `derive.py` and drafts of every section.

What was independently modified, tested, or rejected

Each item below is an AI output that was wrong and was caught by a check, a review or the data:

(1) The first latency benchmark used a *sequential* ring, so the prefetcher turned it into a ∼5 ns "latency". It was replaced by a random Sattolo cycle; the sequential curve is kept only as a contrast (Figure 6).

(2) The first huge-page experiment assumed THP backing because THP was set to `always`. A runtime `smaps_rollup` check showed `AnonHugePages:  0 kB`, so its "huge pages don't help" would have been false. The allocation was rebuilt (2 MiB-aligned mapping, `madvise` before first touch); the grant is now verified for every run, and huge pages give a 1.87× speedup (Section 10).

(3) The converse also happened: under THP `always`, a "4 KiB" buffer could silently get huge pages. `MADV_NOHUGEPAGE` is now set explicitly.

(4) An AI-suggested bandwidth test defaulted to a cache-resident buffer and reported tens of GiB/s as "DRAM bandwidth"; this is now the documented pitfall (Section 11).

(5) The read/write-mix kernel mixed 8 B elements, so every line moved in both directions; mixes are now line-granular (Section 6).

(6) The throughput–latency knee was placed by eye, and Little's Law used the zero-queue latency. Both were replaced by a concurrent loaded-latency probe and a stated knee rule that follows offered load (Section 7).

(7) A "validation" of Little's Law ($L/N \approx 1$) turned out to be an accounting identity. It was replaced by L_{mem} with the probe-measured memory latency.

(8) The scalar unit-stride kernel was quoted as the single-core peak although the strided data contradicted it (Section 5).

(9) A hand-copied table disagreed with its CSV. All tables and quoted numbers are now generated from the data (`derive.py`, `summary.csv`).

(10) Cycle counts were first converted with the in-process add-chain frequency probe, which misreads the clock in about a third of runs (Sections 2 and 12). An independent review

caught this; conversion now uses the fixed 2.8 GHz clock, confirmed with the cycle counter.

Validation

`scripts/validate.sh` asserts the physically necessary orderings (L1 < L2 < L3 < DRAM latency; L1 bandwidth > DRAM; multi-thread > single-thread; loaded > unloaded latency) and agreement of the two latency methods; its output is saved with every experiment. Hardware counters confirm the miss level behind every latency step. Intel MLC was unavailable for an external cross-check; the substitutes and their limits are stated in Sections 3 and 12. Every figure traces to a committed raw/processed file (Section A).

14 Conclusion

The hierarchy behaves as its capacities predict, and the counters show *why* each step occurs. Zero-queue latency is 1.79 ns (5.02 cycles) in L1, 4.76 ns in L2, 18.6 ns in L3 and 109 ns in DRAM, of which 23.1 ns is page walking that 2 MiB pages remove. Concurrency is the other half of the story. Measured under load, latency stays near its unloaded value until 28.8 GiB/s and then climbs steeply towards the 31.6 GiB/s ceiling: past the knee, more cores buy queueing, not bandwidth. With measured latency, Little's Law separates the 54.6 lines the memory system serves at saturation from up to 152 lines that merely queue; one core sustains at most 8.71 random demand misses, but far more lines when the prefetcher streams; and a counter-fitted AMAT model shows random SAXPY paying 14.5 ns per DRAM miss—the DRAM latency divided by 7.5-way overlap. The lessons are about measurement discipline as much as hardware: a bandwidth benchmark must out-size the LLC or it measures cache reuse (Section 11); a latency benchmark must randomize or it measures the prefetcher; a knee must come from measured latency, not throughput alone; a read/write mix must be defined at line granularity; and a huge-page study must verify its pages—in both directions (Section 10).

References

[1] Intel Corporation, *Intel memory latency checker*, Referenced as a recommended tool; not available on the test host.

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: 10.1287/opre.9.3.383

[3] *Perf: Linux profiling with performance counters*. [Online]. Available: https://perf.wiki.kernel.org/

A Reproducibility

Code, scripts, raw and processed data, and this report live in one repository with two top-level directories, `ACS-Code/` and `ACS-Latex/`, at commit `2721691f277e`.

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout 2721691f277e
cd ACS-Code
make tools                                          # tools/summarize
make -C Cache_and_Memory_Performance_Profiling      # mem_latency mem_bandwidth mem_loaded
    kernel_miss
make -C Cache_and_Memory_Performance_Profiling test # physical-ordering sanity checks (validate.
    sh)
```

```
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
Cache_and_Memory_Performance_Profiling/scripts/run.sh zero_queue pattern rw_mix intensity \
    working_set cache_miss tlb pitfall mlc clock_check   # or: run.sh all (CPU_PIN=2 by default)
scripts/sync_data.sh Cache_and_Memory_Performance_Profiling
cd ../ACS-Latex && make memory
```

Table 4: Raw data behind each figure and table (directories under `ACS-Code/Cache_and_Memory_Performance_Profiling/data/raw/`).

Figure / table	`run.sh` experiment	Processed CSV	Raw run directory
Table 2	`zero_queue`	`zero_queue`, `zero_queue_table`	`zero_queue/20261021-004356`
Figures 1 and 2	`pattern`	`pattern_lat`, `pattern_bw`	`pattern/20261021-004356`
Figure 3	`rw_mix`	`rw_mix`, `rw_mix_ref`	`rw_mix/20261021-004356`
Figures 4 and 5, Table 3	`intensity`	`intensity_loaded`, `intensity_mlp`, `littles_law`, `knee`	`intensity/20261021-004356`
Figure 6	`working_set`	`working_set`	`working_set/20261021-004356`
Figure 7	`cache_miss`	`cache_miss_ctr`, `amat_fit`	`cache_miss/20261021-004356`
Figure 8	`tlb`	`tlb_chase`, `tlb_saxpy`	`tlb/20261021-004356`
Figure 9	`pitfall`	`pitfall`	`pitfall/20261021-004356`
Section 3 (MLC)	`mlc`	—	`mlc/20261021-004356`
Sections 2 and 12 (clock)	`clock_check`	— (`summary.csv`)	`clock_check/20261021-194648`

Each raw directory holds the raw rows (one per repetition, with full configuration metadata), `env.txt`, `build.txt`, `commands.txt` and `validate.txt` (Section 2). The processed CSVs are produced from them by `tools/summarize`; `scripts/derive.py` computes the derived tables and `summary.csv`, from which every number in the text is typeset; and `scripts/sync_data.sh` copies all of them into this report's `data/`. The nine experiment directories come from a single `run.sh all` invocation (2026-10-21, 00:43–00:55; one shared timestamp). The `clock_check` diagnostic of Section 12 was run separately the same day (`clock_check/20261021-194648`).